

## v3.0

AI-Powered Distributed Endpoint Monitoring and Remote Scanning Platform

Full Project Documentation — Architecture, Client Connection Lifecycle,  
API Reference, Data Models, Security, Deployment & Operations

Generated August 07, 2026

# 1. Project Overview

System Scanner Pro is an AI-powered distributed endpoint monitoring and remote scanning platform. It pairs a centralized Django admin server with lightweight Python client agents that run on every managed machine. Agents collect hardware and software inventory, stream real-time heartbeats, monitor system events (USB, files, processes, software), and execute scheduled or on-demand scans. The admin dashboard provides fleet-wide visibility with live WebSocket updates, change detection, alerts, predictive analytics, and report generation.

## Key Capabilities

- **Distributed scanning** — one admin server, any number of client agents, cross-platform (Windows / Linux / macOS).
- **Real-time communication** — WebSocket command channel with automatic HTTP polling fallback.
- **Connection lifecycle management** — registration, approval, heartbeat monitoring, blocking, soft-delete and re-registration. See section 3.
- **Event monitoring** — USB, file, process and software change detection with offline disk queue.
- **Scheduled scanning** — APScheduler-driven interval/daily/weekly/monthly scans with offline queue for unreachable devices.
- **AI-ready analytics** — anomaly detection, feature store, and predictive failure-risk scoring.
- **Reporting** — fleet / device / alert reports in PDF and CSV.
- **Security** — JWT auth, HMAC-signed agent traffic, API keys, RBAC and full audit logging.

# 2. Architecture

```
ADMIN SERVER (Django)
|-- REST API (DRF) |-- WebSocket Consumer
|-- Event Bus (pub/sub) |-- APScheduler
|-- Anomaly Detection |-- Feature Store
|-- Predictive Analytics |-- Reports (PDF/CSV)
|-- SQLite / PostgreSQL
^ TCP (HTTP + WebSocket) ^ UDP (discovery, port 45000)
CLIENT AGENTS (1..N)
|-- Scanner (HW/SW) |-- Event Monitors (USB/File/Proc/SW)
|-- Offline Queue (disk) |-- Exponential Backoff
|-- Heartbeat Watchdog |-- WebSocket client (real-time cmds)
|-- UDP discovery listener
```

## Components

| Met<br>hod | Endpoint | Purpose                                                                                          |
|------------|----------|--------------------------------------------------------------------------------------------------|
|            | admin/   | Django admin server: REST API, WebSocket consumers, scheduler, analytics, reports, templates.    |
|            | client/  | Standalone agent: communicator, discovery, fingerprint, key management, scanner, event monitors. |
|            | api/     | Serverless entry point (Vercel) used for cloud registration/discovery.                           |
|            | build/   | PyInstaller build scripts producing admin and client executables.                                |
|            | deploy/  | Deployment helper scripts (Vercel / serverless).                                                 |

## 3. Client Connection Lifecycle

The connection lifecycle is the core of the platform. A machine moves through **creation** (registration) → **approval** → **monitoring** (heartbeat / WebSocket) and finally **deletion**. Both sides — admin server and client agent — coordinate through a well-defined REST + WebSocket contract.

### 3.1 Connection Creation (Registration)

On first run the client generates a registration key and a hardware fingerprint (persisted to **client\_key.json**), discovers the admin URL (UDP broadcast, cloud discovery, or manual entry), and calls `POST /api/register`. The server logic is in `admin/scanner_api/views.py` → `RegisterClientView`.

```
POST /api/register
{
  "registration_key": "ABC123...", # unique per installation
  "hostname": "DESKTOP-7X",
  "platform": "Windows",
  "client_version": "1.0.0",
  "device_fingerprint": "A1B2..." # motherboard+CPU+disk+MAC hash
}
200 -> {"status": "ok", "auto_approved": true|false}
201 -> created; 403 -> key marked deleted (reinstall required)
```

Registration resolution order: (1) if the key already exists the record is refreshed (hostname, IP, fingerprint, last-seen) and the previous approved state is preserved; (2) if the same hardware fingerprint already exists under a different key, the device is **re-registered** by re-pointing the old record to the new key (survives reinstall); (3) otherwise a brand-new **Client** row is created with status **pending** (or **online** if auto-approve is on).

Ownership: newly registered clients inherit the **company** and **owner** of the admin server client record, so fleet data stays partitioned per company.

### 3.2 Approval

Unless auto-approve is enabled in Settings, a registered client stays **pending** until an admin approves it. Approval flips `approved=True`, sets status online and assigns ownership. It can be done individually, in bulk, or automatically.

| Method | Endpoint                               | Purpose                                      |
|--------|----------------------------------------|----------------------------------------------|
| POST   | /api/approve                           | Approve a single client by registration_key. |
| POST   | /api/approve-multiple                  | Bulk approve list of registration_keys.      |
| POST   | /api/monitoring/devices/<uuid>/approve | Approve via the monitoring module.           |
| GET    | /api/clients/<key>/status              | Client-side poll: "approved" or "pending".   |

Auto-approve is controlled by the `auto_approve` Setting (Settings page or `PUT /api/settings`).

### 3.3 Ongoing Connection (Heartbeat + WebSocket)

Once approved, the client maintains the connection through two parallel channels:

- **HTTP heartbeat** — every 30 seconds `POST /api/ping` (or `/api/monitoring/agent/heartbeat-public`). Refreshes `last_seen`, `last_ip`, `hostname` and `version`, and sets status online. Carries CPU/RAM/disk metrics.
- **WebSocket channel** — `ws://HOST/ws/agent/<agent_id>/`. Authenticated with HMAC signature + timestamp; used for real-time commands (`scan_now`, `config_update`) and event streaming. Falls back to HTTP polling if the upgrade is rejected (e.g. serverless).

Offline behaviour: if the admin is unreachable the client applies exponential backoff with jitter, persists events to a disk queue, and replays them on reconnect. The heartbeat watchdog restarts a crashed heartbeat thread automatically. A device

is flagged **offline** when no heartbeat arrives within the stale threshold (default 120 s).

### 3.4 Connection Deletion

Deletion is a **soft delete**: the server sets `deleted=True` so the device vanishes from dashboards while its historical scans/audit trail remain for compliance. The deletion also publishes a `DEVICE_DELETED` event on the event bus (broadcast to live dashboards) and records an activity-log entry.

| Method | Endpoint                             | Purpose                                                   |
|--------|--------------------------------------|-----------------------------------------------------------|
| DELETE | /api/clients/<key>                   | Delete a single client (soft delete).                     |
| POST   | /api/clients/delete-multiple         | Bulk soft-delete list of <code>registration_keys</code> . |
| POST   | /api/monitoring/devices/<uuid>/block | Block a device (deny re-registration/approval).           |

After deletion, a re-registration attempt with the same key is rejected with 403 ("Client has been removed. Reinstall required."). The fingerprint re-registration path is ignored once `deleted=True`, so the operator must un-block or re-register deliberately.

Deletion is not a hard database purge. To permanently remove a device, delete the row directly or perform a full factory reset (section 8).

### 3.5 Connection Settings & Reset

The server can publish its own connection URL and token that client agents use for auto-configuration.

| Method | Endpoint                 | Purpose                                                       |
|--------|--------------------------|---------------------------------------------------------------|
| GET    | /api/settings/connection | Return stored admin server URL + connection token.            |
| POST   | /api/settings/connection | Explicitly generate a new URL + token (never auto-generates). |
| PUT    | /api/settings/connection | Regenerate only the connection token.                         |

- **Reset admin bind IP** — `python admin/main.py --reset`.
- **Reset a client admin URL** — delete `client_config.json` and re-run; or edit `admin_url` directly.
- **Re-register a client** — delete `client_key.json`; a new key is generated and the fingerprint re-links the device to the existing record.
- **Full reset** — delete the DB (`admin/data/scanner.db`) plus client key/config files, then restart both sides.

## 4. Feature Set

### 4.1 Core Platform

- **Hardware scanning** — CPU, RAM, storage, motherboard, GPU, OS, network, software, peripherals, antivirus, Windows updates and user accounts (Windows/Linux/macOS).
- **Admin dashboard** — real-time overview with stats, charts, search/filter and bulk actions.
- **Change detection** — automatic diff between consecutive scans with change alerts.
- **Client management** — approval workflow, heartbeat monitoring, stale detection, grouping.

### 4.2 Real-Time (WebSocket)

- Live dashboard pushes device updates, alerts and health changes instantly.
- Agent command channel (`scan_now`, `config_update`).

- Event broadcasting of hardware/software changes to all connected dashboards.
- Graceful auto-reconnect on both server and client.

### 4.3 Client Event Monitoring

| Method    | Endpoint         | Purpose                                                |
|-----------|------------------|--------------------------------------------------------|
| 5 S       | USB Monitor      | Device insertion/removal (cross-platform).             |
| REAL-TIME | File Monitor     | Watchdog on critical system paths / Windows Event Log. |
| 10 S      | Process Monitor  | New/terminated/suspicious processes.                   |
| 60 S      | Software Monitor | Installed/removed application changes.                 |

Events are batched and delivered over WebSocket (preferred) or HTTP (fallback). Offline events are persisted to disk and replayed on reconnect.

### 4.4 Scheduled Scanning & AI

- APScheduler with interval, daily, weekly, monthly or once schedules.
- Offline scan queue — pending scans delivered when the device comes back.
- **Anomaly detection** — z-score ( $2.5\sigma$ ), IQR ( $1.5\times$ ), trend (50%), static thresholds and compound signals.
- **Predictive analytics** — disk-full time, failure-risk score (0-100), 30/60/90-day capacity forecast.
- **Feature store** — 20+ ML-ready features extracted per device.

## 5. REST API Reference

All endpoints below are relative to the admin base URL (e.g. `http://SERVER`).

### 5.1 Client Registration & Communication

| Method | Endpoint                  | Purpose                                            |
|--------|---------------------------|----------------------------------------------------|
| POST   | /api/register             | Register / refresh a client connection (creation). |
| POST   | /api/approve              | Approve a single client.                           |
| POST   | /api/approve-multiple     | Bulk approve clients.                              |
| POST   | /api/ping                 | Client heartbeat (online status + metrics).        |
| GET    | /api/clients/<key>/status | Check approval status.                             |

### 5.2 Client Deletion & Management

| Method | Endpoint           | Purpose                          |
|--------|--------------------|----------------------------------|
| GET    | /api/clients       | List non-deleted clients.        |
| GET    | /api/clients/<key> | Full client detail + scan diff.  |
| DELETE | /api/clients/<key> | Soft-delete a client connection. |

| Met<br>hod   | Endpoint                        | Purpose                               |
|--------------|---------------------------------|---------------------------------------|
| POST         | /api/clients/delete-multiple    | Bulk soft-delete clients.             |
| PUT          | /api/clients/<key>/manual       | Update manual fields.                 |
| GET<br>/POST | /api/clients/<key>/addons       | List / add addon devices.             |
| DELETE       | /api/clients/<key>/addons/<id>  | Remove an addon device.               |
| GET<br>/PUT  | /api/clients/<key>/scan-config  | Get / update scan interval + enabled. |
| POST         | /api/clients/<key>/scan-now     | Trigger an immediate scan.            |
| GET          | /api/clients/<key>/scan-results | Latest scan result.                   |

### 5.3 Scan Management

| Met<br>hod | Endpoint          | Purpose                               |
|------------|-------------------|---------------------------------------|
| POST       | /api/scan         | Submit scan data from a client.       |
| POST       | /api/scan/local   | Scan the admin server machine.        |
| POST       | /api/scan/all     | Trigger scan on all approved clients. |
| GET        | /api/scan/history | List historical scans.                |

### 5.4 Monitoring Module

| Met<br>hod | Endpoint                               | Purpose                                   |
|------------|----------------------------------------|-------------------------------------------|
| POST       | /api/monitoring/agent/register         | Agent (client) registration.              |
| POST       | /api/monitoring/agent/heartbeat        | HMAC-signed agent heartbeat.              |
| POST       | /api/monitoring/agent/heartbeat-public | Public heartbeat (registration key only). |
| POST       | /api/monitoring/agent/inventory        | Submit hardware/software inventory.       |
| GET        | /api/monitoring/agent/version-check    | Check for newer agent versions.           |
| GET        | /api/monitoring/agent/pending-scans    | Fetch queued scans for an agent.          |
| GET        | /api/monitoring/dashboard              | Aggregate dashboard stats.                |
| GET        | /api/monitoring/devices                | List monitored devices.                   |
| GET        | /api/monitoring/devices/<uuid>         | Device detail.                            |

| Met<br>hod | Endpoint                                  | Purpose                               |
|------------|-------------------------------------------|---------------------------------------|
| POST       | /api/monitoring/devices/<uuid>/approve    | Approve device.                       |
| POST       | /api/monitoring/devices/<uuid>/block      | Block device (connection revocation). |
| GET        | /api/monitoring/devices/<uuid>/metrics    | Device metrics.                       |
| GET        | /api/monitoring/devices/<uuid>/history    | Device history.                       |
| GET        | /api/monitoring/devices/<uuid>/alerts     | Device alerts.                        |
| GET        | /api/monitoring/devices/<uuid>/hardware   | Hardware inventory.                   |
| GET        | /api/monitoring/devices/<uuid>/software   | Software inventory.                   |
| GET        | /api/monitoring/devices/<uuid>/heartbeats | Heartbeat history.                    |
| GET        | /api/monitoring/alerts                    | List all alerts.                      |
| POST       | /api/monitoring/alerts/<uuid>/action      | Ack / resolve / dismiss alert.        |
| POST       | /api/monitoring/devices/bulk              | Bulk device actions.                  |

## 5.5 Scheduled Scanning

| Met<br>hod             | Endpoint                                 | Purpose                           |
|------------------------|------------------------------------------|-----------------------------------|
| GET                    | /api/monitoring/schedules/status         | Scheduler runtime status.         |
| GET                    | /api/monitoring/schedules/pending        | Pending scans.                    |
| GET<br>/POST           | /api/monitoring/schedules                | List / create schedules.          |
| GET<br>/PUT<br>/DELETE | /api/monitoring/schedules/<uuid>         | Get / update / delete a schedule. |
| POST                   | /api/monitoring/schedules/<uuid>/toggle  | Enable or disable a schedule.     |
| GET                    | /api/monitoring/schedules/<uuid>/history | Schedule run history.             |

## 5.6 Reports

| Met<br>hod | Endpoint                          | Purpose            |
|------------|-----------------------------------|--------------------|
| GET        | /api/monitoring/reports/fleet/pdf | Fleet summary PDF. |
| GET        | /api/monitoring/reports/fleet/csv | Fleet CSV export.  |

| Met<br>hod | Endpoint                                      | Purpose            |
|------------|-----------------------------------------------|--------------------|
| GET        | /api/monitoring/reports/device/<u<br>uid>/pdf | Device detail PDF. |
| GET        | /api/monitoring/reports/device/<u<br>uid>/csv | Device CSV export. |
| GET        | /api/monitoring/reports/alerts/pd<br>f        | Alert history PDF. |
| GET        | /api/monitoring/reports/alerts/cs<br>v        | Alert history CSV. |

## 5.7 Authentication & Security

| Met<br>hod       | Endpoint                  | Purpose                             |
|------------------|---------------------------|-------------------------------------|
| POS<br>T         | /api/auth/login           | Session login.                      |
| POS<br>T         | /api/auth/logout          | Session logout.                     |
| GET              | /api/auth/me              | Current user profile.               |
| GET<br>/PU<br>T  | /api/auth/profile         | Get / update profile.               |
| POS<br>T         | /api/auth/change-password | Change password.                    |
| POS<br>T         | /api/auth/upload-avatar   | Upload profile avatar.              |
| GET              | /api/auth/login-history   | Login history.                      |
| GET              | /api/auth/audit-logs      | Audit log trail.                    |
| GET              | /api/auth/active-sessions | Active sessions.                    |
| POS<br>T         | /api/auth/token/obtain    | Obtain JWT access + refresh tokens. |
| POS<br>T         | /api/auth/token/refresh   | Refresh the access token.           |
| POS<br>T         | /api/auth/token/verify    | Validate a token.                   |
| GET<br>/PO<br>ST | /api/auth/api-keys        | List / create API keys.             |
| DEL<br>ETE       | /api/auth/api-keys/<id>   | Revoke an API key.                  |
| GET              | /api/admin/users          | List admin users.                   |
| POS<br>T         | /api/admin/users          | Create admin user.                  |
| DEL<br>ETE       | /api/admin/users/<id>     | Delete admin user.                  |
| GET              | /api/admin/stats          | System statistics.                  |

## 5.8 Settings & Organizations

| Method        | Endpoint                    | Purpose                                                         |
|---------------|-----------------------------|-----------------------------------------------------------------|
| GET /PUT      | /api/settings               | Global settings (auto-approve, stale threshold, scan interval). |
| GET /PUT      | /api/settings/organization  | Organization profile.                                           |
| GET /PUT      | /api/settings/security      | Security settings.                                              |
| GET /PUT      | /api/settings/notifications | Notification settings.                                          |
| GET /POST/PUT | /api/settings/connection    | Connection URL + token management.                              |
| GET /PUT      | /api/settings/dashboard     | Dashboard widget settings.                                      |
| GET           | /api/activity-log           | Recent activity log.                                            |
| GET           | /api/groups                 | List client groups.                                             |
| POST          | /api/groups                 | Create a client group.                                          |
| DELETE        | /api/groups/<id>            | Delete a client group.                                          |

## 5.9 Organization Modules (Locations / Departments / Employees)

| Method          | Endpoint                        | Purpose                           |
|-----------------|---------------------------------|-----------------------------------|
| GET /POST       | /api/locations                  | List / create locations.          |
| GET /PUT/DELETE | /api/locations/<uuid>           | Get / update / delete a location. |
| GET             | /api/locations/<uuid>/dashboard | Location dashboard.               |
| GET /POST       | /api/departments                | List / create departments.        |
| GET /PUT/DELETE | /api/departments/<uuid>         | Department CRUD.                  |

| Met<br>hod             | Endpoint              | Purpose                       |
|------------------------|-----------------------|-------------------------------|
| GET<br>/POST           | /api/employees        | List / create employees.      |
| GET<br>/PUT<br>/DELETE | /api/employees/<uuid> | Employee CRUD.                |
| GET                    | /api/assignments      | Asset assignments.            |
| GET                    | /api/org/audit-logs   | Organization audit logs.      |
| GET                    | /api/org/stats        | Organization dashboard stats. |

## 5.10 Asset Management

| Met<br>hod             | Endpoint                    | Purpose                      |
|------------------------|-----------------------------|------------------------------|
| GET<br>/POST           | /api/assets                 | List / create assets.        |
| GET<br>/PUT<br>/DELETE | /api/assets/<uuid>          | Asset CRUD.                  |
| POST                   | /api/assets/<uuid>/assign   | Assign asset to an employee. |
| POST                   | /api/assets/<uuid>/return   | Return an assigned asset.    |
| POST                   | /api/assets/<uuid>/transfer | Transfer asset.              |
| POST                   | /api/assets/<uuid>/retire   | Retire asset.                |
| POST                   | /api/assets/<uuid>/dispose  | Dispose asset.               |
| GET                    | /api/assets/<uuid>/history  | Asset history.               |
| GET                    | /api/assets/<uuid>/qr       | Asset QR code.               |
| GET                    | /api/assets/dashboard       | Asset dashboard.             |
| GET                    | /api/assets/analytics       | Asset analytics.             |
| GET                    | /api/executive-analytics    | Executive analytics.         |
| GET                    | /api/global-search          | Global search.               |

## 5.11 WebSocket Endpoints

| Met<br>hod | Endpoint                       | Purpose                                           |
|------------|--------------------------------|---------------------------------------------------|
| WS         | ws://HOST/ws/dashboard/        | Admin dashboard real-time updates (fleet events). |
| WS         | ws://HOST/ws/agent/<agent_id>/ | Agent command/control channel (HMAC auth).        |

## 6. Data Models

### 6.1 Client & Fleet (scanner\_api)

| Field                                  | Type  | Purpose                                                                                 |
|----------------------------------------|-------|-----------------------------------------------------------------------------------------|
| Company                                | Model | Organization; isolates fleet data between tenants.                                      |
| ClientGroup                            | Model | User-defined grouping for clients.                                                      |
| Client                                 | Model | A registered device: key, fingerprint, approval, status, heartbeat fields, scan config. |
| ScanResult                             | Model | One scan payload per client; source of change detection.                                |
| AddonDevice                            | Model | Secondary devices attached to a client.                                                 |
| ActivityLog                            | Model | Auditable admin actions (approve, delete, scan, settings).                              |
| Setting                                | Model | Key/value global or per-company settings.                                               |
| AdministratorProfile                   | Model | Admin user profile + company binding.                                                   |
| AuditLog / LoginHistory / LoginAttempt | Model | Security and auth audit records.                                                        |
| DeviceFingerprint                      | Model | Hardware fingerprint registry.                                                          |

### 6.2 Monitoring (monitoring)

| Field                      | Type  | Purpose                                                 |
|----------------------------|-------|---------------------------------------------------------|
| DeviceMonitoringInfo       | Model | Monitored device record (agent_id, secret, status).     |
| HardwareInventory          | Model | Structured hardware inventory payload.                  |
| SoftwareInventory          | Model | Installed software snapshot.                            |
| DeviceHeartbeat            | Model | Heartbeat history per device.                           |
| DeviceMetrics              | Model | CPU/RAM/disk metric series.                             |
| DeviceHistory              | Model | Event/history timeline per device.                      |
| DeviceAlert                | Model | Alert record with severity + status.                    |
| AgentVersion / AgentSecret | Model | Agent release tracking and HMAC secrets.                |
| ScheduledScan              | Model | User-defined scan schedule (interval/daily/weekly/...). |
| PendingScan                | Model | Queued scan for an offline device.                      |
| ScanScheduleLog            | Model | Log of schedule executions.                             |

## 7. Security

| Method | Endpoint       | Purpose                                            |
|--------|----------------|----------------------------------------------------|
| JWT    | Authentication | Access/refresh token pairs for API access (PyJWT). |

| Method       | Endpoint            | Purpose                                          |
|--------------|---------------------|--------------------------------------------------|
| API KEYS     | Programmatic access | Rate limiting + optional IP restrictions.        |
| RBAC         | Authorization       | Super admin / admin / viewer permission classes. |
| HMAC         | Agent traffic       | Agent ↔ server messages signed with HMAC-SHA256. |
| SESSIONS     | Admin panel         | Cookie-based Django sessions for the dashboard.  |
| RATE LIMITER | Monitoring          | Per-agent/endpoint rate limiting.                |

- All admin API views enforce company/owner visibility (data isolation).
- Every significant action writes an ActivityLog / audit record.
- Connection tokens are generated with `secrets.token_hex`.
- Secrets live in `.env`; the file is git-ignored.

## 8. Setup & Operations

### 8.1 Quick Start

```
cd admin-client
pip install -r requirements.txt
python admin/main.py # admin server on port 80
python client/main.py # client agent (any machine)
Default admin login: admin / admin123
Dashboard: http://localhost
UDP discovery port: 45000
```

### 8.2 Management Commands

```
python admin/manage.py migrate # apply DB migrations
python admin/manage.py scan_local # scan the admin machine
python admin/manage.py scan_all # trigger scans on all clients
python admin/manage.py stale_checker # mark stale clients offline
python admin/manage.py alert_checker # evaluate offline alerts
python admin/manage.py health_checker # recompute health scores
python admin/manage.py offline_detector # mark offline devices
python admin/manage.py createsuperuser # create a Django superuser
```

### 8.3 Reset Procedures

| Method         | Endpoint                                  | Purpose                         |
|----------------|-------------------------------------------|---------------------------------|
| RESET ADMIN IP | <code>python admin/main.py --reset</code> | Re-prompt for the bind address. |

| Method               | Endpoint                                 | Purpose                                  |
|----------------------|------------------------------------------|------------------------------------------|
| RESET ADMIN PASSWORD | Delete admin/data/scanner.db then re-run | Recreates DB + default admin.            |
| RESET CLIENT         | Delete client/client_key.json            | New key + fingerprint re-link on re-run. |
| RESET CLIENT URL     | Delete client_config.json                | Re-prompts for the admin URL.            |
| FULL FACTORY RESET   | Delete DB + client key/config files      | Brand-new state on both sides.           |

## 8.4 Running as a Service

```
# Windows scheduled task
schtasks /create /tn "SystemScanner" /tr "C:\path\SystemScannerClient.exe http://SERVER"
/sc onstart
# Linux systemd unit /etc/systemd/system/scanner-client.service
[Service]
ExecStart=/usr/bin/python3 /path/to/client/main.py http://SERVER
Restart=always
RestartSec=10
# macOS launchd (~/.Library/LaunchAgents/com.scanner.client.plist)
RunAtLoadKeepAlive
```

## 9. Configuration

### 9.1 Environment Variables

| Field                 | Type         | Purpose                                                    |
|-----------------------|--------------|------------------------------------------------------------|
| DJANGO_SECRET_KEY     | string       | Django secret key (override the insecure default in prod). |
| DJANGO_DEBUG          | bool         | Django debug mode.                                         |
| SUPABASE_DATABASE_URL | postgres URL | Optional PostgreSQL connection (else SQLite).              |
| SUPABASE_URL          | string       | Optional Supabase project URL for cloud discovery.         |

### 9.2 Network Ports

| Method      | Endpoint              | Purpose                                              |
|-------------|-----------------------|------------------------------------------------------|
| 80 (TCP)    | Web + API + WebSocket | Admin dashboard, REST API and WS upgrade.            |
| 45000 (UDP) | Auto-discovery        | Server broadcasts; clients listen to find the admin. |

### 9.3 Admin CLI Options

| Method                  | Endpoint       | Purpose                |
|-------------------------|----------------|------------------------|
| --PORT                  | Server port    | Default 80.            |
| --HOST                  | Bind address   | Prompted on first run. |
| --DEBUG                 | Debug mode     | Enables Django debug.  |
| --USERNAME / --PASSWORD | Default admin  | admin / admin123.      |
| --RESET                 | Re-ask bind IP | —                      |

### 9.4 Database Locations

| Method             | Endpoint                                   | Purpose                         |
|--------------------|--------------------------------------------|---------------------------------|
| DEVIRONMENT        | admin/data/scanner.db                      | SQLite file next to the server. |
| WINDOWS (PACKAGED) | %APPDATA%\SystemScannerPro\scanner.db      | Per-user app data.              |
| LINUX (PACKAGE)    | ~/.local/share/SystemScannerPro/scanner.db | XDG data home.                  |

| Method                                   | Endpoint                                                  | Purpose          |
|------------------------------------------|-----------------------------------------------------------|------------------|
| MA<br>CO<br>S (P<br>AC<br>KA<br>GE<br>D) | ~/Library/Application Support/SystemScannerPro/scanner.db | App support dir. |

## 9.5 Building Executables

```

pip install pyinstaller
python build/build.py all # build admin + client
python build/build.py admin # build admin only
python build/build.py client # build client only
python build/build.py clean # clean build artifacts
SystemScannerAdmin.exe --port 8080 --username admin --password mypass
SystemScannerClient.exe http://192.168.1.100:80

```

## 10. Deployment

### 10.1 Vercel / Serverless

- `vercel.json` + `api/index.py` expose the Django app as a serverless function.
- WebSocket is unavailable on Vercel — clients automatically fall back to HTTP polling.
- Use PostgreSQL/Supabase (via `SUPABASE_DATABASE_URL`) since the file system is ephemeral.
- `python deploy_vercel.py` assists with deployment.

### 10.2 nginx Reverse Proxy (Production HTTPS)

```

server { listen 80; server_name scanner.yourcompany.com; return 301
https://$host$request_uri; }
server { listen 443 ssl; server_name scanner.yourcompany.com;
ssl_certificate /etc/letsencrypt/live/scanner.yourcompany.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/scanner.yourcompany.com/privkey.pem;
location / { proxy_pass http://127.0.0.1:80; proxy_set_header Host $host; proxy_set_header
X-Forwarded-Proto $scheme; }
location /ws/ { proxy_pass http://127.0.0.1:80; proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade";
proxy_set_header Host $host; proxy_read_timeout 86400; } }
sudo certbot --nginx -d scanner.yourcompany.com
python admin/main.py --host 127.0.0.1 --port 80

```

### 10.3 Cloud Discovery

The admin server can register itself with the cloud discovery registry so remote clients find it without a static IP or domain. Registration data lives in the Supabase/registry backend and is queried by the client during startup (`setup_cloud_discovery.sql` + `admin/main.py` → `register_with_cloud_discovery`).

## 11. Troubleshooting

| Endpoint                                                                                  | Purpose |
|-------------------------------------------------------------------------------------------|---------|
| Verify server running, firewall allows port 80 TCP + 45000 UDP, and server binds 0.0.0.0. |         |

| Endpoint                                                                         | Purpose |
|----------------------------------------------------------------------------------|---------|
| Offline or URL missing http:// prefix.                                           |         |
| Check port 80; if behind nginx ensure /ws/ proxy headers; check browser console. |         |
| Approve from dashboard or enable auto-approve in Settings.                       |         |
| The "Live" badge must be green on the monitoring page; server needs WS enabled.  |         |
| Run python admin/manage.py migrate.                                              |         |
| Verify "[OK] 4 event monitors active" in client output.                          |         |

## 12. Appendix

### 12.1 Scan Data Collected

| Field           | Type     | Purpose                                                            |
|-----------------|----------|--------------------------------------------------------------------|
| Processor       | Snapshot | Manufacturer, model, serial, cores, threads, speed, cache.         |
| RAM             | Snapshot | Per-module manufacturer, capacity, serial, frequency, form factor. |
| Storage         | Snapshot | Disks (model, serial, size, interface) + partitions (fs, mount).   |
| Motherboard     | Snapshot | Manufacturer, model, serial, BIOS version.                         |
| GPU             | Snapshot | Name, vendor, dedicated memory.                                    |
| OS              | Snapshot | Name, version, build, architecture, install date.                  |
| Network         | Snapshot | Interfaces (name, MAC, IPv4, status).                              |
| Peripherals     | Snapshot | Keyboards, mice, audio, webcams, printers, USB devices.            |
| Software        | Snapshot | Installed apps (name, version, publisher).                         |
| Windows Updates | Snapshot | KB IDs + descriptions (Windows only).                              |
| Antivirus       | Snapshot | AV products + firewall status (Windows only).                      |
| User Accounts   | Snapshot | Local user accounts.                                               |

### 12.2 Device Status Indicators

| Method  | Endpoint   | Purpose                                            |
|---------|------------|----------------------------------------------------|
| ONLINE  | Green dot  | Heartbeats received (every 30 s).                  |
| OFFLINE | Red dot    | No heartbeat for over the stale threshold (120 s). |
| PENDING | Yellow dot | Registered but not yet approved.                   |
| BLOCKED | Purple dot | Blocked by admin (connection revoked).             |

### 12.3 Event Bus Event Types

hw\_component\_added, hw\_component\_removed, hw\_component\_modified, sw\_installed, sw\_removed, sw\_version\_changed, sw\_unauthorized, sw\_antivirus\_removed, health\_level\_changed, health\_score\_updated, alert\_created/acknowledged/resolved/dismissed, device\_registered, device\_status\_changed, device\_approved, device\_blocked, device\_deleted, device\_offline, device\_online, agent\_version\_changed, heartbeat\_received, scan\_completed, scan\_scheduled.

---

This document was generated from the [project source and configuration](#). Connection creation and deletion behaviour is defined in [admin/scanner\\_api/views.py](#) (RegisterClientView, ClientDetailView.delete, DeleteMultipleView) and the client [client/communicator.py](#).